

Fronting the Azure AI Stack

with Azure API Management

Gateway pattern for Azure AI services

A practical configuration & troubleshooting walkthrough

Udaiappa Ramachandran (Udai)

CTO | CSO @ Akumina Inc. | Microsoft MVP (AI)

Web: <https://udai.io>

LinkedIn: <https://linkedin.com/in/udair>

Meetup: <https://meetup.com/nashuaug>



Agenda

1 Why front AI services with a gateway

2 Request flow: the mental model for every error

3 Configuring models (Azure OpenAI / Foundry)

4 Paths, policies, authentication & CORS

5 AI gateway policies: tokens, caching, safety

6 The AI Gateway tier & the multi-vendor world

7 Beyond chat: Document Intelligence

8 Validation, troubleshooting & security

The Problem: Islands of Intelligence

Calling AI services directly

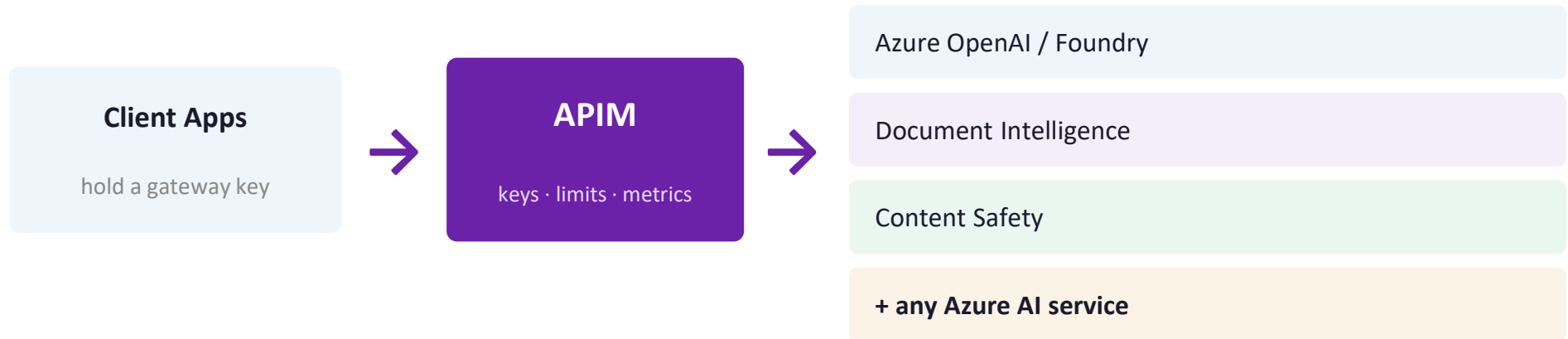
- Every app talks to every provider on its own
- Provider keys copied into each application
- No shared throttling, quotas, or cost view
- Swapping or scaling a backend means client changes
- Browser apps hit CORS and secret-exposure issues

What that costs you

- Secrets sprawl — rotation touches every client
- One noisy app can exhaust a provider's quota
- Usage is unattributable across teams
- No single place for safety or governance
- Each new model or service repeats the wiring

The endpoints multiply, the keys multiply, and governance has nowhere to live.

The Fix: One Governed Entry Point



- Single endpoint; the gateway holds every provider credential
- Enforce subscription keys, token limits, and usage metrics centrally
- Swap or scale a backend with no client change
- Apps carry a revocable gateway key — never a provider secret

Two Ways to Front AI on Azure

Classic APIM instance

- Managed from the Azure portal
- XML policies, API URL suffix, defined operations
- Full control: rewrite-uri, backends, custom policy
- Fronts models, Document Intelligence, any REST AI API
- Generally available across tiers

AI Gateway tier

- Standalone portal at ai.gateway.azure.com
- Models registered as assets — no XML, no suffix
- Routing by model name, policies as portal cards
- Multi-vendor: OpenAI, Anthropic, Bedrock, Vertex...
- Public preview — capabilities and portal UI may change

Same goals — a governed front door for AI — on two surfaces. The AI Gateway tier is in public preview: preview features may change before GA, so pilot with non-critical workloads.

How a Request Flows — The Key to Every Error

1

Operation matching (frontend) APIM matches host + API suffix + operation template. No match → 404 before any policy runs.

2

Backend forwarding (policy) The matched operation's inbound policy runs — set-backend-service, rewrite-uri, limits — then forwards to the backend.

Because policy runs only after matching, a policy can never rescue a path that matched no operation.

404 = a path problem (undefined operation, wrong suffix). 401/403 usually indicate auth; 5xx may be policy, gateway, deployment, or provider.

Configuring Models through APIM

- Add the API with the Azure AI Foundry / Azure OpenAI option — not plain HTTP
- **The import does three things automatically:**
 - Creates operations from the Foundry OpenAPI (e.g. POST /openai/v1/chat/completions)
 - Registers a backend and wires auth — managed identity or stored credential
 - Adds a starter policy: token metrics + a tokens-per-minute limit
- On Client compatibility, choose Azure OpenAI v1 → produces the /openai/v1 base path

Base URL `https://<apim-host>.azure-api.net/<suffix>/openai/v1`

Paths & Operations — 404 and the Doubled Path

- APIM routes only to defined operations — an undefined path 404s at the gateway
- Add the missing operation, or a wildcard, and map it with rewrite-uri at operation scope
- Backend base-url AND request path both containing chat/completions → ../chat/completions/chat/completions

```
<!-- target operation, Inbound -->
<inbound>
  <base />
  <rewrite-uri
    template="/openai/v1/chat/completions" />
</inbound>
```

The rule

Put the path segment in ONE place — the operation template or the backend URL — never both. Use the Trace tab to see the exact backend URL APIM built.

404 vs a doubled path are the two most common wiring mistakes — both are path problems, not policy.

Authentication & the Header Gotcha

The subscription key

- Apps authenticate to the gateway with a subscription key
- The AI Foundry import often names the header api-key — not the default Ocp-Apim-Subscription-Key
- Always confirm the name on the API's Settings tab
- Issue one subscription per app per environment

Why calls get 401

- SDKs default to Authorization: Bearer <key>
- APIM never sees a key in api-key → 401
- Send the subscription key explicitly under the expected header
- Never confuse the gateway key (client) with the provider key (stays in the gateway)

The header name is the single most common cause of a rejected call.

CORS for Browser Clients

- Browsers send a preflight OPTIONS before the real POST
- A cors policy at the API level answers the preflight before the backend — curl and the Test blade won't reveal the failure
- Declare your app origins; allow OPTIONS + POST; allow api-key, Authorization, Content-Type
- Wildcard origin only with allow-credentials=false — list explicit origins in production

```
<cors allow-credentials="false">
  <allowed-origins>
    <origin>https://app.contoso.com</origin></allowed-origins>
  <allowed-methods>
    <method>OPTIONS</method><method>POST</method></allowed-methods>
  <allowed-headers>
    <header>api-key</header><header>Authorization</header>
    <header>Content-Type</header></allowed-headers>
</cors>
```

AI Gateway Policies

`llm-token-limit`

TPM & token quotas per key, IP, or custom counter — pre-checks prompt tokens

`llm-content-safety`

Moderate prompts via Azure AI Content Safety before the backend

`backend load balancer`

Round-robin, weighted, priority, or session-aware across backends

`llm-semantic-cache-*`

Vector-similarity cache of completions — fewer backend calls, lower cost

`llm-emit-token-metric`

Per-consumer token metrics with custom dimensions to Azure Monitor

`backend circuit breaker`

Trips on failures using the backend's Retry-After for recovery

Policies evaluate before the backend is called — so they cap spend as well as risk.

Observability & Governance

What you can see

- Token metrics per model and per consumer
- Request volume, latency, and errors
- Built-in analytics dashboard in the gateway
- Prompts & completions to Azure Monitor (opt-in)

How to correlate

- OpenTelemetry GenAI semantic conventions:
gen_ai.operation.name, gen_ai.request.model,
gen_ai.response.model, gen_ai.token.type
- Send x-correlation-id to tie a gateway log to an app trace
- Never put secrets or prompt content in the correlation id

Payload logging is off by default — prompts and documents carry PII. Enable it deliberately.

The AI Gateway Tier — Models as Assets

- Standalone portal at ai.gateway.azure.com — Entra sign-in, not the Azure portal
- Import Foundry models by discovery — scan subscriptions; no endpoints or keys entered by hand
- Backend auth is key-based or managed identity (the Foundry User role)
- Routing is by the model value in the request body — not by path
- Applications never hold a provider secret — only a gateway runtime access key

```
Runtime endpoint  https://<gateway>.azure-api.net/default/models/openai/v1
```

Naming Is a Public Contract

Permanent by design

- The gateway name is part of the URL — it cannot be renamed
- A model name must be unique across the whole gateway
- Renaming a model or provider isn't exposed in preview
- Set names correctly at import, or delete and re-add

A convention that ages well

- `<org>-aigw-<workload>-<env>-<region>`
- Keep it short enough to read inside a URL
- Don't embed a vendor in the gateway name
- Switching a model's vendor needs no client change if the name stays

The model name is what every client sends — treat it as an API contract, not a label.

Multi-Vendor Reach — Publishable Providers

One gateway, many vendors: each provider is registered as a model asset, so clients keep calling the same model name.

Azure-native

Microsoft Foundry · Azure OpenAI — imported by discovery; managed identity or stored key

Direct SaaS APIs

OpenAI · Anthropic — registered with a provider API key that never leaves the gateway

Google

Vertex AI · Gemini — the Gemini OpenAI-compatible endpoint is the simplest path in

AWS & self-hosted

Amazon Bedrock · any OpenAI-compatible endpoint you run yourself

Backend Auth Caveats by Vendor

Long-lived keys

OpenAI and Gemini (AI Studio) fit the static-header model cleanly

Anthropic

The client must send an anthropic-version header on every call

Google Vertex

OAuth tokens expire (~1h) — prefer the Gemini OpenAI-compatible endpoint

AWS Bedrock

SigV4 can't be a static header — use Bedrock API keys (rotate) or a SigV4-signing proxy

The gateway sends a static header — any vendor needing a signed or expiring credential needs a workaround before it fits.

Beyond Chat: Document Intelligence through APIM

- A separate resource and API surface — configured as its own APIM API
- Deliberately thin: a wildcard frontend path and two API-level policies
- Inbound policy injects the resource key into Ocp-Apim-Subscription-Key
- The wildcard passes path + query through unchanged — every model and route is covered
- Long-running: POST returns 202 + Operation-Location; the client GETs it until "succeeded"

```
<inbound>
  <base />
  <set-header name="Ocp-Apim-Subscription-Key" exists-action="override">
    <value>{{docintel-key}}</value></set-header>
</inbound>
```

The Operation-Location Rewrite

- The 202 response carries Operation-Location built from the resource's own hostname
- A client that follows it verbatim leaves the gateway → 401 (it holds a gateway key, not the resource key)
- An outbound policy rewrites the host to the gateway — the whole analyze-and-poll cycle stays inside it
- A choose guard leaves poll and error responses untouched; use Replace (a bare < breaks the policy XML)

```
<outbound>
  <base />
  <choose><when condition='@(context.Response.Headers.ContainsKey("Operation-Location"))'>
    <set-header name="Operation-Location" exists-action="override">
      <value>@( ...Replace(resourceHost, gatewayHost) )</value></set-header>
    </when></choose>
</outbound>
```

Validate with curl — Test the Gateway Alone

- Prove the gateway independently before wiring an application to it
- A 200 completion confirms four things at once: the key is valid, the model name resolves, no policy blocked the call, and the backend credential works
- For Document Intelligence, use -i to read headers: expect 202 + an Operation-Location on your gateway host
- Send file bytes with --data-binary (never -d); on cmd.exe wrap the body in double quotes and escape them

```
curl -X POST "https://<host>.azure-api.net/<suffix>/openai/v1/chat/completions" \  
  -H "api-key: <SUBSCRIPTION-KEY>" -H "Content-Type: application/json" \  
  -d '{"model": "gpt-4o", "messages": [{"role": "user", "content": "hi"}]}'
```

Troubleshooting — Status Code → Stage

401 / 403

The gateway — bad subscription key, or the backend credential

429

A policy — a token or rate limit was hit

400

Content safety, a malformed body, or the provider (watch cmd.exe quoting)

5xx

Often the provider or the deployment — confirm with the APIM trace

Isolate fast: call the provider directly with the same body · enable the APIM trace · send x-correlation-id and look it up in monitoring

Security Baseline

Keys & identity

- Treat every key as a secret — never in source, logs, or tickets
- One key per app per environment; rotate on exposure
- Prefer managed identity for backend auth — no stored secret, revoke via role
- Back named values with Key Vault for auto-refresh

Guardrails & data

- Content safety + an IP filter as a baseline on every model
- Token limits on the expensive models to contain cost
- Payload logging off by default — documents carry PII
- Data residency spans the whole path: gateway, backend, logs, callers
- Disable direct public access to providers where possible — make the gateway the only path

Policies run before the backend — a blocked request costs nothing at the provider.

Key Takeaways

- **One gateway gives consistent auth, governance, and observability across the AI stack**
- Learn the request-flow stages — they diagnose the large majority of errors
- Keep provider secrets in the gateway; clients only ever hold a revocable gateway key
- The pattern extends the same way: models, Document Intelligence, Content Safety, any Azure AI service
- Validate with curl and let the status code point you at the failing stage

Thank you

Questions & discussion

Udaiappa Ramachandran (Udai)

CTO | CSO @ Akumina Inc. | Microsoft MVP (AI)

Web: <https://udai.io>

LinkedIn: <https://linkedin.com/in/udair>

Meetup: <https://meetup.com/nashuaug>

AI gateway capabilities —

LLM policy reference —

Import an Azure OpenAI API —

